

SQL

BASIC SELECT STATEMENT

Select statement selects a set of attributes ($A_1, A_2, \dots, A_n$) from a set of relations ($R_1, R_2, \dots, R_m$), satisfying a particular condition.

Select $A_1, A_2, \dots, A_n$
From $R_1, R_2, \dots, R_m$
Where condition

Demo: Simple college admission database.

College(cName, state, enrollment)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

College

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkely	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

Student

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800

8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

Apply

	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

EXAMPLE

(i) Students with GPA > 3.6

```
select sID, sName, GPA
from Student
where GPA > 3.6;
```

- (ii) Produce the names of students and the majors they applied to

```
select sName, major  
from Student, Apply  
where Student.sID = Apply.sID;
```

- (iii) Without duplicate, produce (ii)

```
select distinct sName, major  
from Student, Apply  
where Student.sID = Apply.sID;
```

- (iv) Produce the names, GPAs and decisions on applications of students' whose high school size is less than 1000 and who applied to CS at Stanford.

```
select sName, GPA, decision  
from Student, Apply  
where Student.sID = Apply.sID and sizeHS < 1000  
and major = 'CS' and cName = 'Stanford';
```

- (v) Find all large campuses that have someone applying to that campus for CS.

```
select cName  
from College, Apply  
where College.cName = Apply.cName and enrollment > 2000  
and major = 'CS';
```

Note:

The cName in the SELECT clause is ambiguous because both College and Apply relations have columns for cName so, we have to explicitly specify which table owns the cName despite the fact that they are equal:

```
select College.cName  
from College, Apply  
where College.cName = Apply.cName and enrollment > 2000  
and major = 'CS';
```

To avoid duplicates:

```
select distinct College.cName
```

```
from College, Apply  
where College.cName = Apply.cName and enrollment > 2000  
and major = 'CS';
```

- (vi) Get the college name and the number of enrollments of the colleges, the sID, sName and GPA of the students applying to the respective colleges.

```
select Student.sID, sName, GPA, Apply.cName, enrollment  
from Student, College, Apply  
where Apply.sID = Student.sID and Apply.cName = College.cName;
```

- (vii) produce the result of (vi) to be sorted by descending GPA

```
select Student.sID, sName, GPA, Apply.cName, enrollment  
from Student, College, Apply  
where Apply.sID = Student.sID and Apply.cName = College.cName  
= Apply.cName;  
order by GPA desc;
```

The default setting of the ORDER BY clause is the ascending order.

- (viii) In addition to (vii) sort by ascending enrollment

```
select Student.sID, sName, GPA, Apply.cName, enrollment  
from Student, College, Apply  
where Apply.sID = Student.sID and Apply.cName = College.cName  
= Apply.cName;  
order by GPA desc, enrollment;
```

- (ix) All students who are applying for a major that has to do with bio.

```
select sID, major  
from Apply  
where major like '%bio%';
```

- (x) Get all the attributes of (ix)

```
select *  
from Apply  
where major like '%bio%';
```

- (xi) Produce all the attributes of the cross-product of the student and college.

```
select *
```

from Student, College;

- (xii) Produce all the attributes of the Student relation but scale the students' GPA by the $\frac{\text{sizeHS}}{1000}$

select sID, sName, GPA, sizeHS, GPA * $\left(\frac{\text{sizeHS}}{1000}\right)$
from Student;

- (xiii) Reproduce (xii) but name the scaled GPA column "scaledGPA".

select sID, sName, GPA, sizeHS, GPA * $\left(\frac{\text{sizeHS}}{1000}\right)$ as scaledGPA
from Student;

EXAMPLE

- (i) The query below involves all three relations; joins the three relations on their shared attributes and then selects certain information.

select Student. sID, sName, GPA, Apply. cName, enrollment
from Student, College, Apply
where Apply. sID = Student. sID and Apply. cName = College. cName;

- (ii) Use table variables to run the same query in (i)

select S. sID, sName, GPA, A. cName, enrollment
from Student S, College C, Apply A
where A. sID = S. sID and A. cName = C. cName = A. cName;

Table variables make the query more readable. It produces same result.

- (iii) Produce all pairs of students who have the same GPA.

select S1. sID, S1. sName, S1. GPA, S2. sID, S2. sName, S2. GPA
from Student S1, Student S2
where S1. GPA = S2. GPA;

- (iv) Produce (iii) such that a pair contains two different students and every student is paired with a different student.

```
select S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA  
from Student S1, Student S2  
where S1.GPA = S2.GPA and S1.ID <> S2.ID;
```

But this query will still produce a pair and its commutation. E.g.,

Amy	Doris
Doris	Amy

- (v) Produce (iv) such that the output does not have the commutation of a pair.

```
select S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA  
from Student S1, Student S2  
where S1.GPA = S2.GPA and S1.sID < S2.sID;
```

- (vi) Generate a list that includes names of colleges together with names of students.

```
select cName from College  
union  
select sName from Student;
```

- (vii) Produce (vi) such that the schema on both sides of the union is unified and the label of the output columns are renamed as 'name'.

```
select cName as name from College  
union  
select sName as name from Student;
```

If we want to have the duplicates in our result, we add the ALL keyword to the UNION:

- (i) Produce same list as in (vii) but with all the duplicates

```
select cName as name from College  
union all  
select sName as name from Student;
```

- (viii) Produce same list as in (viii) but sort it in an ascending order.

```
select cName as name from College  
union all  
select sName as name from Student  
order by name;
```

- (ix) Get the IDs of all students who have applied to major in CS and major and EE.

```
select sID from Apply where major = 'CS'  
intersect  
select sID from Apply where major = 'EE';
```

- (x) Without using the intersect operator, produce the same result (x).

```
select A1.sID  
from Apply A1, Apply A2  
where A1.sID = A2.sID and A1.major = 'CS' and A2.major = 'EE';
```

- (xi) Get rid of the duplicates from query (xi)

```
select distinct A1.sID  
from Apply A1, Apply A2  
where A1.sID = A2.sID and A1.major = 'CS' and A2.major = 'EE';
```

- (xii) Produce a list of students who applied to CS but not EE

```
select sID from Apply where major = 'CS'  
except  
select sID from Apply where major = 'EE';
```

- (xiii) Rewrite the difference query in (xiii) without using the EXCEPT operator.

```
select A1.sID  
from Apply A1, Apply A2  
where A1.sID = A2.sID and A1.major = 'CS' and A2.major <> 'EE';
```

- (xiv) Remove duplicates from (xiv)

```
select distinct A1.sID  
from Apply A1, Apply A2  
where A1.sID = A2.sID and A1.major = 'CS' and A2.major <> 'EE';
```

EXAMPLE

- (i) Find the IDs and names of students who have applied to major in CS at some college.

```
select sID, sName  
from Student  
where sID in (select sID from Apply where major = 'CS');
```

Output: 5 rows

- (ii) Produce the same result as in (i) without using a subquery in the WHERE clause.

```
select sID, sName  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

- (iii) The query above results in an ***error of disambiguating attributes***: The ID in the SELECT clause could have come from Student or Apply relation; and even though the values are equal, we have to disambiguate by specifying which relation owns the respective attribute;

```
select Student.sID, sName  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

- (iv) Remove the duplicate values from (iii):

```
select distinct Student.sID, sName  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

- (v) Produce only the names of students who applied to major in CS.

```
select sName  
from Student  
where sID in (select sID from Apply where major = 'CS');
```

We get the same result (5 rows) as the previous query. (The two Craig values are actually different students.)

- (vi) Produce the result of (v) using a join instead of using a subquery.

```
select sName
```

```
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

Output: 7 rows

- (vii) Remove the duplicates in (vi)

```
select distinct sName  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

This results in four rows instead of five. The reason is that in the previous query (iv), we included the sID in the result so, the two instances of Craig were two different Craigs (indicated by their sID) hence did not cause duplicates to be eliminated. However, this present query only considers the names, so the two copies of Craig turn into one result.

- (viii) Produce the GPAs of students who are applying to major in CS at some college.

(We are interested in doing some analysis of the GPAs of students who chose to apply to CS. Duplicates are important in order to get the average GPA of CS applicants.)

```
select GPA  
from Student  
where sID in (select sID from Apply where major = 'CS');
```

Output: 5 rows

- (ix) Produce the same result as (viii) using a join instead of a subquery

```
select GPA  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS';
```

Output: 7 rows. This is because some students applied multiple times to CS.

The problem is that if we use this result of rows to compute the average GPA of students who applied to CS, we will count some students more than once – presumably, this is not our intention. Presumably, we want to count each student who applied to CS once in computation. This worked in the previous query which had a subquery

So, if we run this query below

```
select distinct GPA  
from Student, Apply  
where Student.sID = Apply.sID and major = 'CS');
```

- (x) Find students who have applied to major in CS but have not applied to major in EE.

```
select sID, sName  
from Student  
where sID in (select sID from Apply  
              where major = 'CS') and sID not in (select sID from Apply  
                                                  where major = 'EE');
```

Output: 3 students who have applied to major in CS but have not applied to major in EE.

- (xi) Alternatively, we can write the query as 'NOT sID IN (SELECT...)' rather than SID NOT IN (SELECT ...)

```
select sID, sName  
from Student  
where sID in (select sID from Apply  
              where major = 'CS') and not sID in (select sID from Apply  
                                                  where major = 'EE');
```

- (xii) Find all colleges such that some other college is in the same state.

```
select cName, state  
from College C1  
where exists (select * from College C2  
              where C2.state = C1.state);
```

Output: Wrong answer that lists all the colleges

This is because when the query is runs, C1 and C2 could be bound to the same college. So, every college is in the same state as another college if the other college could be the same college.

To correct this, we further condition add a condition, inside the subquery, that says that C1 and C2 are different colleges.

```
select cName, state  
from College C1
```

```
where exists (select * from College C2  
where C2.state = C1.state  
and C1.cName <> C2.cName;
```

- (xiii) Use subqueries and NOT EXISTS operator to find the college that has the largest enrollment.

```
select cName, state  
from College C1  
where not exists (select * from College C2  
where C2.enrollment > C1.enrollment;
```

Output: Berkeley

- (xiv) Find the student with the highest GPA.

```
select sName, GPA  
from Student S1  
where not exists (select * from Student S2  
where S2.GPA > S1.GPA);
```

Output: 4 rows because there is a tie in GPA among four students.

- (xv) Write the same query as in (xiv) without using a subquery.

```
select S1.sName, S1.GPA  
from Student S1, Student S2  
where S1.GPA > S2.GPA;
```

Output: is wrong

Add the DISTINCT keyword.

```
select distinct S1.sName, S1.GPA  
from Student S1, Student S2  
where S1.GPA > S2.GPA;
```

Output: wrong

- (xvi) Use the ALL keyword to find the student with the highest GPA

```
select sName, GPA  
from Student  
where GPA >= all (select GPA from Student);
```

- (xvii) Rewrite (xvii) using <>

```
select sName, GPA  
from Student S1  
where GPA > all (select GPA from Student S2  
                  where S2.sID <> S1.sID);
```

Output: nothing

- (xviii) Modify query (xvii) to look for colleges that have the highest enrollment.

```
select cName  
from College C1  
where enrollment > all (select enrollment from College C2  
                  where C2.cName <> C1.cName);
```

Output: Berkeley

- (xix) Use the ANY keyword to run the same query posed in (xviii).

```
select cName  
from College C1  
where not enrollment <= any (select enrollment from College C2  
                  where C2.cName <> C1.cName);
```

Output: Berkeley

- (xx) Find all students who are not from the smallest high school

```
select sID, sName, sizeHS  
from Student  
where sizeHS > any (select sizeHS from Student);
```

This returns a student if there exists another student whose high school size is smaller.

- (xxi) Rewrite query (xx) without using ANY

```
select sID, sName, sizeHS  
from Student S1  
where exists (select * from Student S2 where S2.sizeHS < S1.sizeHS);
```

- (xxii) Find all students that applied to major in CS but not EE.

```
select sID, sName  
from Student  
where sID = any (select sID from Apply where major = 'CS')  
and not sID = any(select sID from Apply where major = 'EE');
```

EXAMPLE

- (i) Produce the list of students and the course they applied to major in.

```
select distinct sName, major  
from Student, Apply  
where Student.sID = Apply.sID;
```

- (ii) Rewrite query (i) using an equivalent theta join, which is called INNER JOIN in SQL.

```
select distinct sName, major  
from Student inner join Apply  
on Student.sID = Apply.sID;
```

- (iii) Find the name and GPA of students who came from a high school with less than 1000 students and applied to major in CS at Stanford.

```
select sName, GPA  
from Student, Apply  
where Student.sID = Apply.sID and sizeHS < 1000  
and major = 'CS' and cName = 'Stanford';
```

- (iv) Rewrite query (iii) using join operator. (Effectively, we'll replace the comma with join. the comma being the cross-product.)

```
select sName, GPA  
from Student join Apply  
on Student.sID = Apply.sID  
where sizeHS < 1000  
and major = 'CS' and cName = 'Stanford';
```

- (v) We can actually put all WHERE conditions in query (iv) into the ON clause so that the ON clause becomes the AND or conjunction of all the three conditions:

```
select sName, GPA  
from Student join Apply  
on Student.sID = Apply.sID  
and sizeHS < 1000  
and major = 'CS' and cName = 'Stanford';
```

- (vi) Find a bunch of information about students' applications.

```
select Apply.sID, sName, GPA, Apply.cName, enrollment  
from Apply, Student, College  
where Apply.sID = Student.sID and Apply.cName = College.cName;
```

- (vii) Rewrite query (vi) using the join operator. (So, we'll change commas to joins and WHERE conditions to ON since they are the two conditions that are actually joining Apply with Student and Apply with College.)

```
select Apply.sID, sName, GPA, Apply.cName, enrollment  
from Apply join Student join College  
on Apply.sID = Student.sID and Apply.cName = College.cName;
```

Output: Syntax error

- (i) Change query (vii) to be a binary join [by joining Apply and Student first, and then joining College; and move the conditions to their appropriate positions]

```
select Apply.sID, sName, GPA, Apply.cName, enrollment  
from (Apply join Student on Apply.sID = Student.sID) join College  
on Apply.cName = College.cName;
```

EXAMPLE

- (i) Compute the average GPA of students in the database.

```
select avg(GPA)
from Student;
```

- (ii) Find the lowest GPA of students applying to CS

```
select min(GPA)
from Student, Apply
where Student.sID = Apply.sID and major = 'CS';
```

This query is a little bit complicated, as it applies a join.

For clarification: select * to see the students who have applied to CS

- (iii) Average GPA of students who applied to CS

```
select avg(GPA)
from Student, Apply
where Student.sID = Apply.sID and major = 'CS';
```

Output: 3.7

Note:

In fact, this last result (on average) is possibly not precisely what we are looking for. In our previous lesson

- (iv) In order to do that, we use the subquery form, where we select from Student and then we just check for each student whether their IDs are among those who applied to CS

```
select *
from Student
where sID in (select sID from Apply where major = 'CS');
```

Output: 5 rows; one instance of such student.

So, when we compute the average GPA, we'll correctly count each student's GPA once.

```
select avg(GPA)
```

```
from Student  
where sID in (select sID from Apply where major = 'CS');
```

Output: 3.68

Count function: just returns the number of tuples in the result.

- (v) Find the number of colleges in our database whose enrollment is greater than 15000.

```
select count(*)  
from College  
where enrollment > 15000;
```

Output: 2

- (vi) Find the number of students who applied to Cornell.

```
select count(*)  
from Apply  
where cName = 'Cornell';
```

Output: 6

- (vii) In the count function, we can put a special keyword DISTINCT and then the name of one or more attributes. In this example, the COUNT will look at the result and then count distinct values for the particular attribute:

```
select count(distinct sID)  
from Apply  
where cName = 'Cornell';
```

Output: 3

- (viii) Find all students such that the number of other students with same GPA as the student is equal to the number of other students with the same sizeHS.

```
select *  
from Student s1  
where (select count(*) from Student s2  
       where s2.sID <> s1.sID and s2.GPA = s1.GPA) =  
       (select count(*) from Student s2 where s2.sID <> s1.sID  
         and s2.sizeHS = s1.sizeHS);
```

Output: 6 rows

Verify from the original Student table.

- (ix) Calculate the amount by which average GPA of students who applied to CS exceeds average of students who did not apply to CS.

```
select CS.avgGPA – NonCS.avgGPA  
from (select avg(GPA)as avgGPA from Student  
where sID in(select sID from Apply where major = ' CS'))  
as CS,  
(select avg(GPA)as avgGPA from Student  
where sID not in (select sID from Apply  
where major = ' CS'))as NonCS;
```

Output: 0.194

- (x) Rewrite (ix) using subqueries in the SELECT clause. (Remember that in the previous lesson, we learned that we can write a subquery in a SELECT clause as long as it returns a single value.

```
select distinct (select avg(GPA) as avgGPA from Student  
where sID in (  
(select sID from Apply where major = 'CS'))  
– (select avg(GPA)as avgGPA from Student  
where sID not in (select sID from Apply  
where major = ' CS'))as d  
from Student;
```

- (xi) Find the number of applicants to each college

```
select cName, count(*)  
from Apply  
group by cName;
```

For clarification: select * from Apply order by cName;

- (xii) Find the total enrollment of college students for each state.

```
select state, sum(enrollment)  
from College  
group by state;
```

- (xiii) Calculate the minimum and maximum GPAs of applicants to each college and their major.

```
select cName, major, min(GPA), max(GPA)
from Student, Apply
where Student.sID = Apply.sID
group by cName, major;
```

For clarification:

```
select cName, major, GPA
from Student, Apply
where Student.sID = Apply.sID
order by cName, major;
```

- (xiv) Find the spread of GPA (i.e., the difference between the minimum GPA and max(GPA) for each college.

Solution: Take the whole query in (xiii) and put it into the FROM clause; call it M. Then, access the aggregate in the SELECT clause.

```
select mx – mn
from (select cName, major, min(GPA) as mn, max(GPA) as mx
      from Student, Apply
      where Student.sID = Apply.sID
      group by cName, major) M;
```

The output with zero are the combinations that only have one student.

- (xv) Find the largest spread of GPA.

```
select max(mx – mn)
from (select cName, major, min(GPA) as mn, max(GPA) as mx
      from Student, Apply
      where Student.sID = Apply.sID
      group by cName, major) M;
```

(xvi) Find the number of colleges applied to by each student.

```
select Student.sID, count(distinct cName)
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID;
```

For clarification:

```
select Student.sID, cName
from Student, Apply
where Student.sID = Apply.sID
order by Student.sID;
```

(xvii) To have sName appear in the result of (xvi)

```
select Student.sID, sName, count(distinct cName)
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID;
```

(xviii) In addition to GROUP BY query (xvii), project the college name.

```
select Student.sID, sName, count(distinct cName), cName
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID;
```

Output in SQLite not Postgres: For instance, for Amy, it shows Stanford as the college name

The original query

```
select Student.sID, sName, count(distinct cName)
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID;
```

that finds the number of colleges applied by each student produces eight rows.

But there might be students in the database who have not applied to any college at all. Such students won't appear in the result because they are not going to be in the results of the join so, we don't see any zero counts for such students.

- (xix) To include the sIDs of students who did not apply to any college [and for such – include a zero in the output]:

```
select Student.sID, count(distinct cName)
from Student, Apply
where Student.sID = Apply.sID
group by Student.sID
union
select sID, 0
from Student
where sID not in (select sID from Apply);
```

- (xx) Find colleges that have fewer than five applications.

```
select cName
from Apply
group by cName
having count(*) < 5;
```

Output: Berkeley and MIT

- (xxi) Rewrite query (xx) without using the HAVING clause and grouping clause.

```
select distinct cName
from Apply A1
where 5 > (select count(*) from Apply A2
where A2.cName = A1.cName);
```

- (xxii) What if we are not interested those colleges that have fewer than five applications but rather, those colleges that have fewer than five applicants.

[It is a subtle point but instead of counting the number of Apply tuples for each college, we count the number of distinct sIDs that appear in the group of Apply tuples for that college.]

```
select cName
from Apply
group by cName
having count(distinct sID) < 5;
```

Output: 3 rows. One more college, Cornell joins the group with more than five applications but has fewer than five applicants.

(xxiii) Find all majors whose applicants' maximum GPA is below the average.

```
select major  
from Student, Apply  
where Student.sID = Apply.sID  
group by major  
having max(GPA) < (select avg(GPA) from Student);
```

Output: bioengineering and psychology

EXAMPLE (NULL VALUES)

- (i) Insert two students Kevin and Lori into the database with null GPA values and the following information (You have to turn off the not-null property of GPA column):

```
insert in Student values (432, 'kevin', null, 1500);  
insert in Student values (321, 'Lori, null, 2500);
```

- (ii) Find students whose GPAs are greater than 3.5

```
select sID, sName, GPA  
from Student  
where GPA > 3.5;
```

Output: doesn't include Kevin and Lori because their GPAs are NULL.

- (iii) Find students with GPA <= 3.5

```
select sID, sName, GPA  
from Student  
where GPA <= 3.5;
```

Output: doesn't include Kevin and Lori

- (iv) Students with GPAs greater than 3.5 or <= to 3.5

```
select sID, sName, GPA
```

```
from Student  
where GPA > 3.5 or GPA <= 3.5;
```

Output: still doesn't include Kevin and Lori because we can determine that their GPAs satisfy either of the conditions.

- (v) Produce the result from query (iv) that includes students with null GPAs.

```
select sID, sName, GPA  
from Student  
where GPA > 3.5 or GPA <= 3.5 or GPA is null;
```

IS NULL is a phrase in SQL that fetches values that are null.

Output: All the students including Kevin and Lori

- (vi) Students with GPA > 3.5 or sizeHS > 1600

```
select sID, sName, GPA  
from Student  
where GPA > 3.5 or sizeHS < 1600;
```

Output: includes Kevin.

Although his GPA is null which makes it impossible to determine that it satisfies the GPA condition, since his high school size is 1500, he satisfies the second part of the condition (or disjunction) which is why he appears in the result.

- (vii) Students with GPA > 3.5 or sizeHS < 1600 or sizeHS >= 1600.

[The second and third part of the condition form a tautology so every student is returned.]

```
select sID, sName, GPA, sizeHS  
from Student  
where GPA > 3.5 or sizeHS < 1600 or sizeHS ≥ 1600;
```

- (viii) Find the number of students whose GPA is not null (for starters).

```
select count(*)  
from Student  
where GPA is not null;
```

Output: 12 students, which is the original number we had before adding Kevin and Lori.

- (ix) Count the distinct GPAs that are not null.

```
select count(distinct GPA)
from Student
where GPA is not null;
```

Output: 7 rows

- (x) Count all distinct GPAs

```
select count(distinct GPA)
from Student
```

Output: 7 rows. Same result as when we included NOT NULL.

Therefore, COUNT DISTINCT does not count NULL values.

- (xi) Produce the list of distinct GPAs of students.

```
select distinct GPA
from Student
```

Output: 8 rows of which the first is the null GPA

Inserting Data

Involves two methods:

- (a) First method allows us to insert one tuple into the database by specifying its actual value.

<pre>Insert Into Table Values(A_1, A_2, ..., A_n)</pre>
--

- (b) Second method is to run a query over the database as a SELECT statement. That SELECT statement will produce a set of tuples, and as long as that set of

tuples as the same schema as the table, we can insert all the tuples into the table.

Insert Into Table
Select-Statement

Deleting Existing Data

Delete From Table
Where condition

Updating Existing Data

Insert Into Table
Values($A_1, A_2, \dots, A_n$)

Updating Multiple Attributes in a tuple

Update Table
Set $A_1 = \text{Expr}_1, A_2 = \text{Expr}_2, \dots, A_n = \text{Expr}_n$
Where Condition

Demo: Simple college admission database.

College(cName, state, enrollment)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

- (i) Add a new college 'AFIT' to our database]

insert into College values('AFIT', 'Kad', 1500);

- (ii) Have all the students who did not apply to anywhere apply to AFIT.

Solution:

- ✓ Let's start by finding those students who have not applied to anywhere.

**select *
from Student
where sID not in (select sID from Apply);**

Output: 4 students

- ✓ Then, turn this query into something that constructs the tuples that we want to insert into the Apply relation. (Remember that Apply relation has sID, cName, major, decision)
So, we'll construct the student ID: we want them to apply to AFIT; major in CS; and for now, we don't know the decision of the application so we assign a null value to the decision:

**select sID, 'AFIT', 'CS', null
from Student
where sID not in (select sID from Apply);**

Output: For our students, we have constructed four tuples that are ready to be inserted into the Apply relation – They have the appropriate schema.

- ✓ So, now that we have that query, we insert the query into the Apply relation:

**insert into Apply
select sID, 'AFIT', 'CS', null
from Student
where sID not in (select sID from Apply);**

For clarification: take a look at the Apply relation.

- (iii) Admit into AFIT department of EE all students who were turned down in EE elsewhere.

Solution:

- ✓ First find students who have applied for EE at another college and were rejected

```
select *  
from Student  
where sID not in (select sID from Apply  
                where major = 'EE' and decision = 'N');
```

Output: two students

- ✓ Now let's turn this query into a query that constructs tuples that we would like to insert into the Apply relation:
We need the sID; and we want this sIDs to apply to AFIT; major in EE; and accept their application, right away:

```
select sID, 'AFIT', 'EE', 'Y'  
from Student  
where sID in (select sID from Apply  
              where major = 'EE' and decision = 'N');
```

Output: two tuples

- ✓ Insert these two tuples into the Apply relation:

```
insert into Apply  
select sID, 'AFIT', 'EE', 'Y'  
from Student  
where sID in (select sID from Apply  
              where major = 'EE' and decision = 'N');
```

Delete

- (iv) Delete all students who applied to more than two different majors

Solution:

- ✓ Start by finding students who applied to more than two majors:

```
select sID, count(distinct major)  
from Apply  
group by sID
```

```
having count(distinct najor) > 2;
```

The query takes the Apply relation; forms partitions based on sIDs. So, it considers the set of applications for each student individually; then it counts how many distinct majors that are in each group – if this number is greater than two, it returns that student's ID and how many majors they applied to.

Output: 2 students who applied for three majors each

- ✓ Since we don't want such students, we will get rid of them. (We don't need to display the count of the majors they applied to):

```
delete from Student  
where sID in  
(select sID  
from Apply  
group by sID  
having count(distinct najor) > 2);
```

Confirmation: Refresh the Student table and you'll see that sIDs 345 and 876 are gone.

- (v) We've deleted them from the Student relation but we haven't deleted them from the Apply relation: We'll write exactly the same deletion as

```
delete from Apply  
where sID in  
select sID  
from Apply  
group by sID  
having count(distinct major) > 2);
```

Note:

Some database systems do not allow such deletion commands where the subquery includes the same relation that you're deletion from. Postgres does allow it.

Confirmation: Refresh the Apply table: 876 and 345 are no more. "Otilo"

EXERCISE

Try this alternative deletion method in MySQL or SQLite.

- (vi) Delete colleges with no CS applicants

Solution:

- ✓ Start with the select command

```
select * from College  
where cName not in (select cName  
  
from Student where GPA < 3.6);
```

Output: Nobody has applied to CS at Cornell

- ✓ Then

```
delete from College  
where cName not in (select cName  
from Apply where major = 'CS');
```

Confirmation: Refresh College. No Cornell record exists.

Update Commands

- (vii) Accept applications to AFIT with GPA < 3.6 but turn them into Economics majors.

Solution:

- ✓ Start with students who have applied to AFIT with GPA < 3.6

```
select * from Apply  
where cName = 'AFIT'  
and sID in (select sID from Student where GPA < 3.6);
```

- ✓ Update the Apply relation with every tuple that satisfy the condition above and accept those students to economics major

```
update Apply  
set decision = 'Y', major = 'economics'  
where cName = 'AFIT'  
and sID in (select sID  
from Student where GPA < 3.6);
```

The next command is a more complicated and highly motivated command:

- (viii) Turn the highest-GPA EE applicant into a CSE (Computer Science and Engineering) applicant.

Solution:

✓ First

```
select * from Apply  
where major = 'EE'  
and sID in (select sID from Student  
where GPA >= all (select GPA from Student  
where sID in (select sID from Apply  
where major = 'EE'))));
```

This query has triply-nested subqueries:

- The innermost subquery finds students who have applied to EE;
- The next [outer] subquery finds the GPA of the students (in the innermost query) who have applied to EE
- The outermost subquery finds all students whose GPA is at least as high as the highest EE applicant
- WHERE clause finds all students who have applied to EE and have the highest GPA.

Output: Student 123 who has applied to EE at Cornell and AFIT

- ✓ Then we update the Apply relation with this result and set the major to CS:

```
update Apply  
set major = 'CS'  
where major = 'EE'  
and sID in (select sID from Student  
where GPA >= all (select GPA from Student  
where sID in (select sID from Apply  
where major = 'EE'))));
```

- (ix) Assign to every student the maximum GPA and the smallest high school size in the database.

[This example, although quite silly, demonstrates that in the SET command, the right-hand-side of the equality can itself, be a subquery.

```
update Student  
set GPA = (select max(GPA)from Student),  
sizeHS = (select min(sizeHS)from Student);
```

This query updates every student. Since there is no WHERE clause, it means that every tuple will satisfy the condition; and the query sets the GPA of each student to the result of finding the maximum GPA in the database; and the query sets the sizeHS for each student to the result of finding the minimum high school size.

Confirmation: Refresh the Student relation

Output: Every student has the GPA = 3.9 and sizeHS = 20000

- (x) Accept every student that applied to a college
- ```
update Apply
set decision = 'Y'
```

Confirmation: Refresh Apply relation. Every student has been accepted.

**X**

---

Stephen Ebuka Iheagwara  
Lecturer

i